

JMigrate Core - Quick Integration Guide

Industry Standard Database Schema Versioning for Java & Spring Boot

1. What is JMigrate Core?

JMigrate Core is a lightweight, high-performance database migration engine for Java. Unlike bloated ORM tools, it scans your classpath for raw SQL scripts, tracks execution history in a dedicated 'schema_version_history' table, and rolls back transactions cleanly if any script fails.

2. Step 1: Add Maven Dependency (pom.xml)

Add the JitPack repository and jmigrate-core dependency to your pom.xml:

```
<repositories>
  <repository>
    <id>jitpack.io</id>
    <url>https://jitpack.io</url>
  </repository>
</repositories>

<dependencies>
  <dependency>
    <groupId>com.github.Aditya-code-2</groupId>
    <artifactId>jmigrate-core-repo1</artifactId>
    <version>v.0.1</version>
  </dependency>
</dependencies>
```

3. Step 2: Create SQL Migration Script

Place your SQL scripts inside 'src/main/resources/db/migration/' following the strict regex format: V<Number>__<Description>.sql (e.g., V1__init_schema.sql).

```
-- File: src/main/resources/db/migration/V1__init_schema.sql
CREATE TABLE IF NOT EXISTS ads (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4. Step 3: Configure in Spring Boot

Create a configuration class that runs the migration engine automatically at startup using @PostConstruct:

```
package com.mycompany.config;

import com.jmigrate.jmigrate_core.engine.JMigrateEngine;
import jakarta.annotation.PostConstruct;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;
```

JMigrate Core - Quick Integration Guide

Industry Standard Database Schema Versioning for Java & Spring Boot

```
@Configuration
public class JMigrateConfig {
    @Value("${spring.datasource.url}") private String url;
    @Value("${spring.datasource.username}") private String username;
    @Value("${spring.datasource.password}") private String password;

    @PostConstruct
    public void runMigrations() {
        JMigrateEngine engine = new JMigrateEngine(url, username, password);
        engine.migrate(); // Discovers and runs pending scripts atomically
    }
}
```

5. How It Works in Production

1. Startup Check: When the app starts, JMigrate connects to the database and checks 'schema_version_history'.
2. Auto-Scan: It scans 'db/migration' for scripts matching V<Version>__<Name>.sql.
3. Safe Execution: If V1 is already applied, it is skipped. New scripts are executed in an atomic transaction.